

ShinobuChat 快速 Demo 复盘

来源文件：shinobuchat-demo-retrospective.md

依据 Git 历史、现有项目结构与 .claude 中的架构记录整理。结论：早期 demo 可以找到，并且演进链路清晰；它不是一个被废弃的草稿，而是当前 ShinobuChat 的产品骨架。

1. Demo 在 Git 中的位置

阶段	提交	时间	关键词	说明
架构草图	c6552709	2026-05-10 21:35	设计架构	新增 flowstate_system_overview.svg、flowstate_message_dataflow.svg、flowstate_memory_layer.svg，先把系统想清楚。
交互原型	a7b7a8a8	2026-05-10 23:44	设计原型图	新增 shinobuchat_prototype.html，形成可点击、可感知的 Live Room 原型。
可运行 MVP	95b8c519	2026-05-11 04:18	Auth / API / SSE / chat-dock	接入 FastAPI、用户/设备/会话/消息模型、POST /messages SSE 流式响应和 Web 消息气泡。
离线与路由 MVP	7179efeb	2026-05-11 14:16	offline cache / routing MVP	增加前端离线缓存、auto/chat/agent 路由状态、乐观消息渲染与 SSE 幂等回放。
Agent 雏形	e575dcf3	2026-05-13 02:26	intent_recognizer / todo API	加入关键词/正则意图识别、Todo 工具、Agent 状态 SSE。
角色与技能	69b1960a	2026-05-13 07:27	builtin skills / character API	加入内置技能、角色 API，开始从聊天 demo 走向 AI 桌面伙伴。

2. 当初快速完成的 Demo 是什么

早期 demo 的核心不是“做一个聊天框”，而是把普通 AI 对话重新放进一个有场景、有角色、有状态反馈的桌面空间里。

a7b7a8a8 中的 shinobuchat_prototype.html 已经具备当前产品的雏形：

- 中央是 Live Room 舞台，用手绘化头像占位表达未来 Live2D 桌宠。
- 底部与侧边同时存在输入区和会话面板，强调“陪伴在场”而不是单纯问答。
- 路由模式从原型阶段就存在：自动决策、纯聊天、强制 Agent。
- 设置抽屉中已经预留任务、记忆中心、角色语气、Live2D、语音、多端入口、隐私撤销等产品方向。
- 状态提示通过 SSE 流式响应中、Agent 空闲、conversation_id 等信息表达，解决 AI 等待过程中的不确定感。

到 95b8c519，这个设计原型被迅速推进成可运行 MVP：

- 后端有 FastAPI 工程骨架和 /api/v1 路由分层。
- 数据层有 users、devices、conversations、messages。
- 认证层有用户注册、设备授权、JWT/OAuth2 风格登录链路。
- 对话层有 POST /messages 和 text/event-stream 响应。
- 前端能创建 demo 用户、保存 conversation_id、读取 SSE、渲染消息气泡。

到 7179efeb，demo 又补上了“真实应用才会痛”的部分：本地缓存、pending sync、client_message_id、消息分类和路由恢复。也就是说，早期 demo 已经开始回答一个关键问题：如果 AI 桌宠是长期陪伴型产品，断网、刷新、重复回放、任务/陪伴分流都不能被当作边角问题。

3. Demo 的思想

这个 demo 的真正价值，在于它从第一天就避开了两个常见误区。

第一，它没有把 AI 当成一个无形的 API。原型把 AI 放进 Live Room、聊天气泡、状态胶囊、会话侧栏、桌宠窗口之中，让用户能感到“她在这里”。这是一种产品判断：陪伴型应用的核心不是模型多强，而是等待时不失联、回应时有情绪、任务推进时有解释。

第二，它没有把 Agent 做成独立后台工具。早期路由模式就把 auto/chat/agent 放在输入区，说明项目从一开始就在处理“什么时候只是陪我聊，什么时候需要替我做事”这个边界。后来的 IntentRecognizer、SkillRegistry、AgentOrchestrator 都是在扩展这个原点。

因此，ShinobuChat 的早期 demo 可以概括为：

一个以桌面陪伴为外壳、以 SSE 流式对话为主干、以路由意图识别为决策层、以任务与记忆为未来扩展方向的 AI-Native 桌宠聊天原型。

4. Demo 到当前成品的演进

Demo 想法	当前成品对应实现	演进结果
Live Room 舞台	frontend/shinobu-chat/src/live2d/Live2DStage.tsx	从 CSS 占位头像演进为 Pixi + Cubism Live2D 模型加载。
会话气泡与历史面板	MessageList.tsx、ConversationList.tsx、message State.ts	从原型 DOM 操作演进为 React 状态管理与单元测试。
SSE 流式响应	app/services/message_service.py、frontend/shinobu-chat/src/api/sse.ts	从简单 chunk 展示演进为 conversation / progress / emotion / audio / done 结构化事件。
路由模式	RouteMode.AUTO / CHAT / AGENT	从 UI 选择项演进为后端消息路径分流。
Agent 与技能	SkillRegistry、AgentOrchestrator、app/services/tools/	从意图规则雏形演进为 SKILL.md 驱动的工具调用框架。
语音功能	voice_service.py、asr_service.py、tts_pipeline.py	从设置项预留演进为 ASR、TTS、分句播放、口型同步链路。
情绪反馈	emotion_parser.py、Live2D emotion mapping	从“角色氛围”演进为 [emotion] 标签驱动的表情、语音与前端状态联动。

Demo 想法	当前成品对应实现	演进结果
隐私与撤销	当前仍是方向性设计	原型中已提出，后续应继续补足记忆撤销、解释与数据导出。

5. 一句话总结

当初快速完成的 demo 并不是一个临时页面，而是一份产品宣言：ShinobuChat 要做的不是“聊天机器人加皮肤”，而是让 AI 以桌面伙伴的形态进入用户日常，在聊天、任务、记忆、语音和情绪之间形成一个连续的陪伴系统。